CEBU INSTITUTE OF TECHNOLOGY UNIVERSITY

College of Computer Studies

SYSTEM DESIGN DOCUMENT

(SDD)

CIT MedConnect

Healthcare Management System for CIT-U

Project Title	CIT MedConnect
Prepared By	Bayonas, Jay Lord C.
Course	IT342 – G2 System Integration and Architecture
Version	2.0
Date	May 27, 2026
Status	Final – Revised

REVISION HISTORY

Ver.	Date	Author	Changes Made	Status
0.1	Jan 15, 2026	Bayonas, J.L.	Initial draft	Draft
0.2	Jan 22, 2026	Bayonas, J.L.	Added API specifications	Review
0.3	Jan 29, 2026	Bayonas, J.L.	Updated database design	Review
0.4	Feb 05, 2026	Bayonas, J.L.	Added UI/UX wireframes	Review
0.5	Feb 12, 2026	Bayonas, J.L.	Incorporated feedback	Revised
1.0	Feb 23, 2026	Bayonas, J.L.	Baseline version for development	Approved
2.0	May 27, 2026	Bayonas, J.L.	Revised and aligned with actual implementation	Final

TABLE OF CONTENTS

EXECUTIVE SUMMARY

Project Overview

CIT MedConnect is a cross-platform healthcare management system developed for the Cebu Institute of Technology University community. The system provides a centralized digital platform that replaces manual clinic operations by offering online appointment booking, role-based medical record management, and real-time notification delivery across both web and mobile clients.

The platform supports three user roles — Student, Staff, and Admin — each with tailored access privileges. Students can book clinic appointments, view their personal medical records, and receive notifications. Staff and Admins manage appointments, create and update medical records after consultations, and send broadcast notifications to users.

CIT MedConnect is built on a three-tier architecture: a Spring Boot 3.5 REST API backend, a React 18 web client, and a native Android mobile application. Supabase (managed PostgreSQL) serves as the primary cloud database and storage backend. GitHub OAuth 2.0 is integrated as an alternative authentication mechanism alongside standard email/password login.

Objectives

1. Develop a fully functional healthcare management MVP encompassing authentication, appointment scheduling, and medical record management.
2. Implement a three-tier architecture using Spring Boot (backend), React (web frontend), and Android (mobile).
3. Design and expose RESTful APIs for all system features, consumed by both web and mobile clients.
4. Enforce role-based access control (RBAC) to ensure data security and appropriate privilege separation.
5. Integrate GitHub OAuth 2.0 as a supplementary login method.
6. Deploy all system components to production-ready cloud environments.

Scope

Included Features

- User registration and login (email/password and GitHub OAuth 2.0)
- Token-based session management
- Role-based access control (Student, Staff, Admin)
- Appointment creation, status management, rescheduling, and cancellation
- Time slot management with availability tracking
- Staff-managed medical record creation, update, and retrieval
- Patient read-only access to personal medical records
- In-app notification delivery (individual and broadcast)
- Responsive React web application

- Native Android mobile application
- Supabase PostgreSQL relational database

Excluded Features

- AI-assisted medical diagnosis
- Multi-hospital or multi-clinic deployment
- Payment processing
- Advanced analytics dashboard
- SMTP email notifications (deferred to future iteration)
- Push notifications (mobile)

1.0 INTRODUCTION

1.1 Purpose

This System Design Document (SDD) serves as the authoritative technical reference for the CIT MedConnect system. It consolidates all design decisions made during development — including architectural choices, API contracts, database schema, UI/UX wireframe references, and implementation workflows — and has been revised to align with the actual codebase hosted at https://github.com/jaylordbayonas/IT342-Bayonas-CitMedConnect.

1.2 Technology Stack

Layer	Technologies
Backend	Java 17, Spring Boot 3.5.x, Spring Security, Spring Data JPA, Hibernate, Spring OAuth2 Client, Gson, BCrypt
Database	Supabase (managed PostgreSQL)
Web Frontend	React 18, JavaScript (JSX), React Router v6, Axios, Lucide React, Vite
Mobile	Android (Kotlin), Activities + ViewModels, LiveData, Retrofit 2, OkHttp, Gson, DataStore, Coroutines, minSdk 26
Build Tools	Maven (Backend), npm / Vite (Web), Gradle (Android)
Deployment	Railway or Render (Backend), Vercel (Web), APK Distribution (Mobile)
Authentication	Token-based session (UUID tokens), GitHub OAuth 2.0
Testing	JUnit 5, Mockito, Testcontainers, REST-assured, Vitest, Testing Library

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name	CIT MedConnect
Domain	Healthcare Management
Primary Users	Students, Clinic Staff, Administrators
Problem Statement	The university clinic relies on manual processes for appointment scheduling and patient record handling, leading to long waiting times, data inconsistency, and inefficient service delivery.
Solution	CIT MedConnect provides a centralized digital platform with online appointment booking, secure role-based medical record management, and in-app notification delivery.

2.2 User Roles

Role	Description
STUDENT	Registered university student. Can book appointments, view personal medical records (read-only), and receive notifications.
STAFF	Clinic staff member. Full appointment management, creates and updates medical records post-consultation, sends notifications.
ADMIN	System administrator. All STAFF privileges plus user management and broadcast notification capabilities.

2.3 Core User Journeys

Journey 1: Student – First-Time Appointment Booking

7. Student visits the web or mobile application.
8. Student registers an account with their school credentials.
9. Student logs in and is redirected to the Dashboard.
10. Student navigates to the Appointments section.
11. Student selects an available time slot, enters a reason, and submits the booking.
12. The system creates the appointment with status PENDING and notifies the student.
13. Staff reviews and confirms or cancels the appointment.

Journey 2: Student – Returning User

14. Student logs in with existing credentials or via GitHub OAuth.
15. Student views upcoming appointments on the Dashboard or Appointments page.
16. Student accesses Medical Records to view past consultation records (read-only).
17. Student checks the Notifications section for updates.

Journey 3: Staff / Admin – Appointment & Record Management

18. Staff member logs in.
19. Staff views all appointments via the Appointments management view.
20. Staff confirms, completes, or cancels appointments as appropriate.
21. After a completed consultation, Staff creates a Medical Record linked to the appointment.
22. Staff may send an individual or broadcast notification to affected patients.
23. Patients can view the newly created record in their Medical Records section.

2.4 Feature List (MoSCoW)

Must Have

- User authentication (register, login, logout)
- Token-based session management
- GitHub OAuth 2.0 login
- Role-based access control (Student / Staff / Admin)
- Appointment CRUD with status lifecycle (PENDING → CONFIRMED → COMPLETED / CANCELLED)
- Time slot creation and availability management
- Staff-managed medical records (CRUD)
- Medical record linked to appointment and patient
- In-app notification system (individual and broadcast)

Should Have

- Appointment rescheduling
- Calendar view of appointments
- Notification read/unread tracking
- Responsive web design
- Input validation with user feedback

Could Have

- Real-time queue updates
- Basic admin dashboard with statistics
- Appointment reminders

Won't Have

- AI diagnosis
- SMTP email notifications
- Multi-language support
- Advanced analytics

2.5 Detailed Feature Specifications

Feature: User Authentication

Screens: Registration, Login, Dashboard redirect

Fields: School ID (optional, auto-generated if omitted), First Name, Last Name, Email, Password, Phone, Role, Gender, Age

Validation: Email uniqueness, password minimum 6 characters, required field checks

API Endpoints: POST /api/auth/register, POST /api/auth/login

Security: BCrypt password hashing, UUID token generation; GitHub OAuth via /api/auth/oauth2/github/callback

Feature: Appointment Management

Functions: Student creates appointment by selecting a time slot; Staff views all appointments; Staff updates appointment status; Student or Staff cancels; Staff reschedules appointment to a new time slot.

Status Lifecycle: PENDING → CONFIRMED → COMPLETED (or CANCELLED at any stage)

API Endpoints: POST /api/appointments/, GET /api/appointments/staff/all, GET /api/appointments/student/my-appointments, PUT /api/appointments/{id}/confirm, PUT /api/appointments/{id}/complete, PUT /api/appointments/{id}/cancel, PUT /api/appointments/{id}/reschedule, DELETE /api/appointments/{id}

Feature: Time Slot Management

Staff creates available time slots with a specific date, start and end time, and maximum booking capacity. The system tracks current booking count and marks a slot unavailable once capacity is reached.

API Endpoints: GET /api/timeslots/available, GET /api/timeslots/staff/{staffId}, PUT /api/timeslots/{id}/book

Feature: Medical Records Management

Staff/Admin creates a medical record after a completed appointment, capturing diagnosis, symptoms, treatment, prescription, vital signs, allergies, medical history, and notes. Records are linked to both the patient (via school_id) and the originating appointment. Patients can only read their own records.

API Endpoints: POST /api/medical-records/, GET /api/medical-records/, GET /api/medical-records/{id}, GET /api/medical-records/user/{userId}, PUT /api/medical-records/{id}, DELETE /api/medical-records/{id}

Feature: Notification System

The system supports targeted notifications sent to individual users as well as broadcast notifications sent to all students, all staff, or everyone. Each notification includes a type, title, message, read status, and global flag.

API Endpoints: POST /api/notifications/send, POST /api/notifications/broadcast/students, POST /api/notifications/broadcast/staff, POST /api/notifications/broadcast/all, GET /api/notifications/user/{schoolId}, PUT /api/notifications/{id}/read

2.6 Acceptance Criteria

AC-1: Successful User Registration

- Given I am a new user
- When I submit a valid email, password (min. 6 characters), and name
- Then my account is created with role STUDENT by default
- And I receive a session token
- And I am redirected to the Dashboard

AC-2: Appointment Booking

- Given I am logged in as a Student
- When I select an available time slot and submit a reason
- Then an appointment record is created with status PENDING
- And the appointment is visible to Staff via the management view

AC-3: Medical Record Creation

- Given I am logged in as Staff
- When I create a medical record linked to a completed appointment
- Then the record is stored in the database with the correct patient and appointment reference
- And the patient can view the record in their Medical Records section

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API response time ≤ 3 seconds for 95% of requests under normal load
- Web page initial load ≤ 3 seconds on broadband connections
- Android application cold start ≤ 4 seconds
- Support at least 100 concurrent users
- Database query execution ≤ 500 ms

3.2 Security Requirements

- HTTPS enforced for all client-server communications in production
- Token-based authentication required for all protected endpoints
- Passwords hashed using BCrypt before persistence
- Role-based endpoint protection (Student cannot access Staff-only endpoints)
- SQL injection prevention via JPA/Hibernate parameterized queries
- Cross-Origin Resource Sharing (CORS) configured; @CrossOrigin(origins = "*") used in development
- Supabase credentials stored in environment variables, never in source code
- Medical record modification restricted to Staff and Admin roles
- Sensitive fields (passwords, OAuth tokens) never exposed in API responses

3.3 Compatibility Requirements

- Web: Modern browsers (Chrome, Firefox, Safari, Edge)
- Mobile: Android API Level 26 and above (Android 8.0+)
- Backend: Java 17 runtime
- Responsive layout across mobile, tablet, and desktop viewports

3.4 Usability Requirements

- Students can complete an appointment booking within 3 minutes
- Consistent navigation patterns across web and mobile
- Clear inline error messages for all form validation failures
- Touch targets minimum 44x44 dp on Android
- Accessible, readable form labels and helper text

4.0 SYSTEM ARCHITECTURE

4.1 Architectural Pattern

CIT MedConnect follows a three-tier layered architecture:

- Presentation Tier: React 18 web application and native Android mobile application
- Application Tier: Spring Boot 3.5 REST API backend
- Data Tier: Supabase managed PostgreSQL database

All client-to-backend communication occurs over HTTP/HTTPS using JSON-formatted request and response payloads. Clients authenticate by including the session token in the Authorization header or the X-User-ID / X-User-Role custom headers, depending on the endpoint type.

4.2 Component Overview

The backend is organized into feature-based packages under edu.cit.bayonas.citmedconnect.features. Each feature module contains its own controller, service, repository, entity, DTO, and mapper layers.

Module	Package	Responsibilities
Authentication	features/auth	User registration, login, user CRUD, password management, role assignment
OAuth2	features/oauth2	GitHub OAuth2 code exchange, user info retrieval, account linking
Appointments	features/appointments	Appointment lifecycle (PENDING → CONFIRMED → COMPLETED/CANCELLED), time slot management
Medical Records	features/medicalrecords	Medical record CRUD, linked to patient and appointment
Notifications	features/notifications	Individual and broadcast notification delivery, read-status management
Security Config	features/shared/config	Spring Security configuration, CORS, endpoint protection rules

4.3 Component Diagram (Textual Description)

The following describes the component relationships. A visual UML component diagram should be rendered from this description using a diagramming tool (e.g., draw.io, PlantUML).

Client Layer

- Web Client (React 18 / Vite): Communicates with the Spring Boot API via Axios. Stores session data in React Context. Pages include Landing, Login, Register, Dashboard, Appointments, Medical Records, Calendar, Notifications, Profile, and OAuth2 Callback.
- Mobile Client (Android / Kotlin): Communicates via Retrofit 2. Persists session data in DataStore Preferences. Activities include Landing, Login, Register, Dashboard, Appointments, Admin Appointments, Medical Records, Notifications, and Profile.

Backend Layer – Spring Boot API (Java 17)

- AuthController → UserService → UserRepository → users table
- OAuth2Controller → OAuth2Service → UserRepository
- AppointmentController → AppointmentService → AppointmentRepository / TimeSlotRepository
- MedicalRecordController → MedicalRecordService → MedicalRecordRepository
- NotificationController → NotificationService → NotificationRepository

External Services

- Supabase PostgreSQL: Primary relational database for all persistent data
- GitHub OAuth 2.0 API: Used for social login via https://github.com/login/oauth/access_token and the GitHub User API

4.4 Data Flow

Standard API Request

24. Client sends HTTP request with Authorization: Bearer <token> or X-User-ID / X-User-Role headers.
25. Spring Security intercepts and validates the request against configured security rules.
26. The appropriate Controller receives the request and delegates to the Service layer.
27. The Service executes business logic and interacts with the Repository for database operations.
28. JPA/Hibernate translates repository calls into SQL queries against Supabase PostgreSQL.
29. The result is mapped to a DTO and returned as a JSON response.

GitHub OAuth2 Flow

30. Client redirects user to GitHub authorization URL.
31. User grants consent; GitHub redirects back with an authorization code.
32. Client sends the code to POST /api/auth/oauth2/github/exchange-code.
33. Backend exchanges the code for a GitHub access token via the GitHub OAuth API.
34. Backend retrieves user info from the GitHub User API.
35. OAuth2Service creates a new user account or links to an existing one.
36. Backend returns a session token and user data to the client.

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Property	Value
Base URL (dev)	http://localhost:8080/api
Base URL (prod)	https://api.citmedconnect.com/api
Format	JSON for all requests and responses
User Identity	X-User-ID and X-User-Role headers, or Authorization: Bearer <token>
Content-Type	application/json

5.2 Standard Response Structure

Most endpoints return a consistent JSON envelope:

```
{ "success": boolean, "data": object | null, "error": { "code": string, "message": string, "details": object | null }, "timestamp": string }
```

5.3 Authentication Endpoints

POST /api/auth/register — Register New User

Method	POST
URL	/api/auth/register
Auth Required	No
Request Body	{ "firstName": "Juan", "lastName": "Dela Cruz", "email": "juan@email.com", "password": "pass123", "schoolId": "21-0001", "phone": "09171234567", "role": "STUDENT", "gender": "MALE", "age": 20 }
Response 201	{ "success": true, "message": "Registration successful", "token": "<uuid-token>", "data": { "schoolId": "21-0001", "firstName": "Juan", "lastName": "Dela Cruz", "email": "juan@email.com", "role": "STUDENT" } }
Response 409	Email already registered

POST /api/auth/login — Authenticate User

Method	POST
URL	/api/auth/login
Auth Required	No
Request Body	{ "email": "juan@email.com", "password": "pass123" }

Response 200	{ "success": true, "message": "Login successful", "token": "<uuid-token>", "data": { "schoolId": "21-0001", "firstName": "Juan", "lastName": "Dela Cruz", "email": "juan@email.com", "role": "STUDENT" } }
Response 401	Invalid email or password

GET /api/auth/users — List All Users (Admin)

Method	GET
URL	/api/auth/users
Auth Required	Yes (Admin)
Response 200	Array of UserDTO objects

PUT /api/auth/users/{schoolId} — Update User

Method	PUT
URL	/api/auth/users/{schoolId}
Auth Required	Yes
Request Body	Updated UserDTO fields
Response 200	Updated UserDTO object

5.4 OAuth2 Endpoints

POST /api/auth/oauth2/github/exchange-code — Exchange Authorization Code

Method	POST
URL	/api/auth/oauth2/github/exchange-code
Auth Required	No
Request Body	{ "code": "github_auth_code", "redirectUri": "http://localhost:5173/oauth2/callback" }
Response 200	{ "accessToken": "gho_xxxxx" }

POST /api/auth/oauth2/github/callback — Process GitHub Login

Method	POST
URL	/api/auth/oauth2/github/callback
Auth Required	No
Request Body	{ "accessToken": "gho_xxxxx" }

Response 200	{ "success": true, "message": "GitHub login successful", "token": "<uuid-token>", "user": { ... }, "isNewUser": false }

5.5 Appointment Endpoints

POST /api/appointments/ — Create Appointment

Method	POST
URL	/api/appointments/
Role	STUDENT
Request Body	{ "user": { "schoolId": "21-0001" }, "timeSlot": { "timeSlotId": 5 }, "reason": "Fever and cough", "status": "PENDING" }
Response 200	AppointmentEntity with appointmentId, status PENDING

GET /api/appointments/staff/all — All Appointments (Staff)

Method	GET
URL	/api/appointments/staff/all
Role	STAFF / ADMIN
Response 200	Array of all AppointmentEntity objects

GET /api/appointments/student/my-appointments — Student Appointments

Method	GET
URL	/api/appointments/student/my-appointments
Role	STUDENT
Query Params	userId (optional; falls back to X-User-ID header or Security context)
Response 200	Array of AppointmentEntity objects for the requesting student

PUT /api/appointments/{id}/confirm — Confirm Appointment

Method	PUT
URL	/api/appointments/{id}/confirm
Role	STAFF / ADMIN
Response 200	Updated AppointmentEntity with status CONFIRMED

PUT /api/appointments/{id}/complete — Complete Appointment

Method	PUT
URL	/api/appointments/{id}/complete
Role	STAFF / ADMIN
Response 200	Updated AppointmentEntity with status COMPLETED

PUT /api/appointments/{id}/cancel — Cancel Appointment

Method	PUT
URL	/api/appointments/{id}/cancel
Role	STUDENT / STAFF / ADMIN
Response 200	Updated AppointmentEntity with status CANCELLED

PUT /api/appointments/{id}/reschedule — Reschedule Appointment

Method	PUT
URL	/api/appointments/{id}/reschedule
Role	STAFF / ADMIN
Request Body	{ "newTimeSlotId": 12 }
Response 200	Updated AppointmentEntity with new time slot

5.6 Time Slot Endpoints

GET /api/timeslots/available — Available Slots

Method	GET
URL	/api/timeslots/available
Auth Required	Yes
Response 200	Array of TimeSlotDTO objects where isAvailable is true

PUT /api/timeslots/{id}/book — Book a Time Slot

Method	PUT
URL	/api/timeslots/{id}/book
Role	STUDENT
Response 200	Updated TimeSlotDTO with incremented currentBookings

5.7 Medical Record Endpoints

POST /api/medical-records/ — Create Medical Record

Method	POST
URL	/api/medical-records/
Role	STAFF / ADMIN
Request Body	{ "userId": "21-0001", "appointmentId": 101, "diagnosis": "Gingivitis", "symptoms": "Gum pain, bleeding", "treatment": "Scaling", "prescription": "Mouthwash twice daily", "vitalSigns": "BP 120/80, Temp 36.5", "allergies": "None", "medicalHistory": "None", "notes": "Follow-up in 1 month", "createdBy": "22-0099" }
Response 201	MedicalRecordDTO with recordId

GET /api/medical-records/user/{userId} — Records by Patient

Method	GET
URL	/api/medical-records/user/{userId}
Role	STUDENT (own records only) / STAFF / ADMIN
Response 200	Array of MedicalRecordDTO for the specified user

GET /api/medical-records/appointment/{appointmentId} — Record by Appointment

Method	GET
URL	/api/medical-records/appointment/{appointmentId}
Role	STAFF / ADMIN
Response 200	MedicalRecordDTO linked to the given appointment

5.8 Notification Endpoints

POST /api/notifications/send — Send to Individual User

Method	POST
URL	/api/notifications/send
Role	STAFF / ADMIN
Request Body	{ "recipientId": "21-0001", "title": "Appointment Confirmed", "message": "Your appointment on June 1 is confirmed.", "type": "APPOINTMENT" }
Response 201	NotificationDTO

POST /api/notifications/broadcast/students — Broadcast to All Students

Method	POST
URL	/api/notifications/broadcast/students
Role	STAFF / ADMIN
Request Body	{ "title": "Clinic Advisory", "message": "The clinic will be closed on June 12." }
Response 201	Array of NotificationDTO (one per student)

GET /api/notifications/user/{schoolId} — User Notifications

Method	GET
URL	/api/notifications/user/{schoolId}
Auth Required	Yes
Response 200	Array of NotificationDTO for the user

PUT /api/notifications/{id}/read — Mark as Read

Method	PUT
URL	/api/notifications/{id}/read
Auth Required	Yes
Response 200	Updated NotificationDTO with isRead = true

5.9 Error Handling

HTTP Code	Error Code	Meaning
200	—	Successful request
201	—	Resource created successfully
400	VALID-001	Bad request / validation failure
401	AUTH-001	Invalid credentials
403	AUTH-003	Insufficient permissions (role check failed)
404	DB-001	Resource not found
409	DB-002	Conflict — duplicate entry (e.g., email already exists)
500	SYSTEM-001	Internal server error

6.0 DATABASE DESIGN

6.1 Overview

CIT MedConnect uses Supabase (managed PostgreSQL) as its primary database. The schema consists of five core tables: users, time_slots, appointments, medical_records, and notification. All tables are managed via Spring Data JPA with Hibernate as the ORM layer.

6.2 Entity Relationship Summary

Relationship	Cardinality	Description
users → appointments	One-to-Many	A user (student) can have multiple appointments
time_slots → appointments	One-to-Many	A time slot can be referenced by multiple appointment records
users → time_slots (staff)	One-to-Many	A staff member can own multiple time slots
users → medical_records	One-to-Many	A user can have multiple medical records
appointments → medical_records	One-to-One (optional)	A completed appointment may have one associated medical record
users → notification	One-to-Many	A user can receive multiple notifications

6.3 Table Definitions

Table: users

Column Name	Data Type	Constraints	Description
school_id (PK)	VARCHAR	NOT NULL, UNIQUE	Primary key. School ID or UUID if not provided.
first_name	VARCHAR	NOT NULL	User first name
last_name	VARCHAR	NOT NULL	User last name
email	VARCHAR	NOT NULL, UNIQUE	Institutional or personal email
phone	VARCHAR	NOT NULL	Contact phone number
password	VARCHAR	NOT NULL	BCrypt-hashed password
oauth_provider	VARCHAR	NULLABLE	OAuth provider name (e.g., github)
oauth_id	VARCHAR	NULLABLE	OAuth provider user identifier
role	VARCHAR	NOT NULL	STUDENT | STAFF | ADMIN

gender	VARCHAR	NOT NULL	MALE | FEMALE | OTHER
age	INTEGER	NOT NULL	User age
created_at	TIMESTAMP	NOT NULL	Account creation timestamp

Table: time_slots

Column Name	Data Type	Constraints	Description
time_slot_id (PK)	BIGINT	GENERATED IDENTITY	Primary key
slot_date	DATE	NOT NULL	Date of the time slot
start_time	TIME	NOT NULL	Slot start time
end_time	TIME	NOT NULL	Slot end time
is_available	BOOLEAN	NOT NULL	Whether the slot is open for booking
max_bookings	INTEGER	NOT NULL	Maximum allowed bookings for this slot
current_bookings	INTEGER	NOT NULL	Current number of bookings
staff_id (FK)	VARCHAR	NULLABLE	References users(school_id) — assigned staff
is_within_business_hours	BOOLEAN		Flag indicating business hours compliance

Table: appointments

Column Name	Data Type	Constraints	Description
appointment_id (PK)	BIGINT	GENERATED IDENTITY	Primary key
school_id (FK)	VARCHAR	NOT NULL	References users(school_id) — booking patient
time_slot_id (FK)	BIGINT	NOT NULL	References time_slots(time_slot_id)
status	VARCHAR	NOT NULL	PENDING | CONFIRMED | COMPLETED | CANCELLED
reason	TEXT	NULLABLE	Patient-provided reason for the appointment
notes	TEXT	NULLABLE	Additional staff notes
created_at	TIMESTAMP	NOT NULL	Record creation timestamp
updated_at	TIMESTAMP		Last update timestamp (@PreUpdate)

Table: medical_records

Column Name	Data Type	Constraints	Description
record_id (PK)	BIGINT	GENERATED IDENTITY	Primary key
school_id (FK)	VARCHAR	NOT NULL	References users(school_id) — patient
appointment_id (FK)	BIGINT	NULLABLE	References appointments(appointment_id)
diagnosis	TEXT		Clinical diagnosis
symptoms	TEXT		Patient-reported symptoms
treatment	TEXT		Treatment administered
prescription	TEXT		Medications prescribed
vital_signs	TEXT		Recorded vital signs
allergies	TEXT		Known allergies
medical_history	TEXT		Relevant past medical history
notes	TEXT		Additional clinical notes
record_date	TIMESTAMP	NOT NULL	Date and time record was created
created_by	VARCHAR		school_id of staff who created the record
created_at	TIMESTAMP	NOT NULL	Persistence creation timestamp
updated_at	TIMESTAMP		Last update timestamp (@PreUpdate)

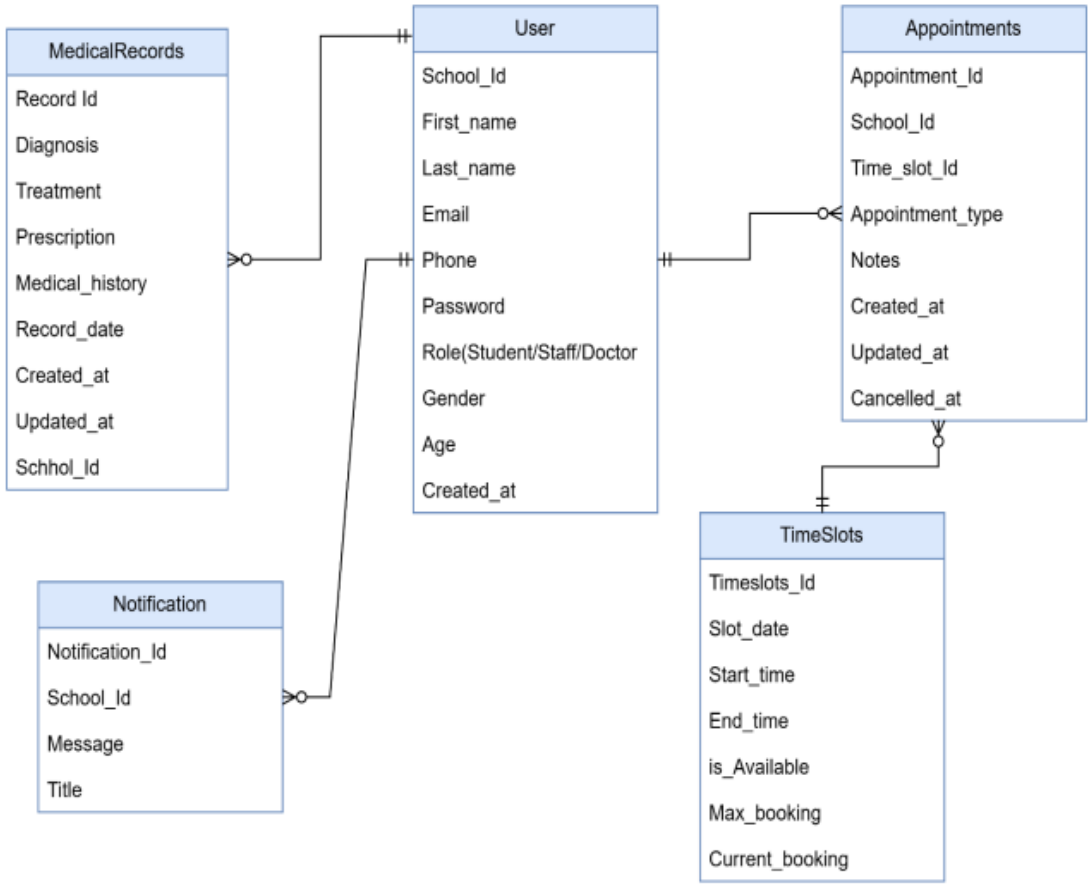
Table: notification

Column Name	Data Type	Constraints	Description
Notification_Id (PK)	VARCHAR	NOT NULL	UUID-based primary key
School_Id (FK)	VARCHAR	NOT NULL	References users(school_id) — recipient
title	VARCHAR	NOT NULL	Notification title
message	TEXT	NOT NULL	Notification message body
Notification_Type	VARCHAR	NOT NULL	APPOINTMENT | MEDICAL_RECORD | STUDENT_BROADCAST | STAFF_BROADCAST | GLOBAL_BROADCAST
is_global	BOOLEAN		True for broadcast notifications

is_read	BOOLEAN	NOT NULL, DEFAULT false	Read status flag
Created_At	TIMESTAMP	NOT NULL	Record creation timestamp (@PrePersist)

6.4 Entity Relationship Diagram

The ERD below represents the five core tables and their relationships. A rendered diagram should be produced using a diagramming tool (e.g., dbdiagram.io, draw.io).



7.0 UI/UX DESIGN

Figma Design Reference:
https://www.figma.com/design/3pIkiaMzQYadHu5xlpJEh5/CitMedConnent

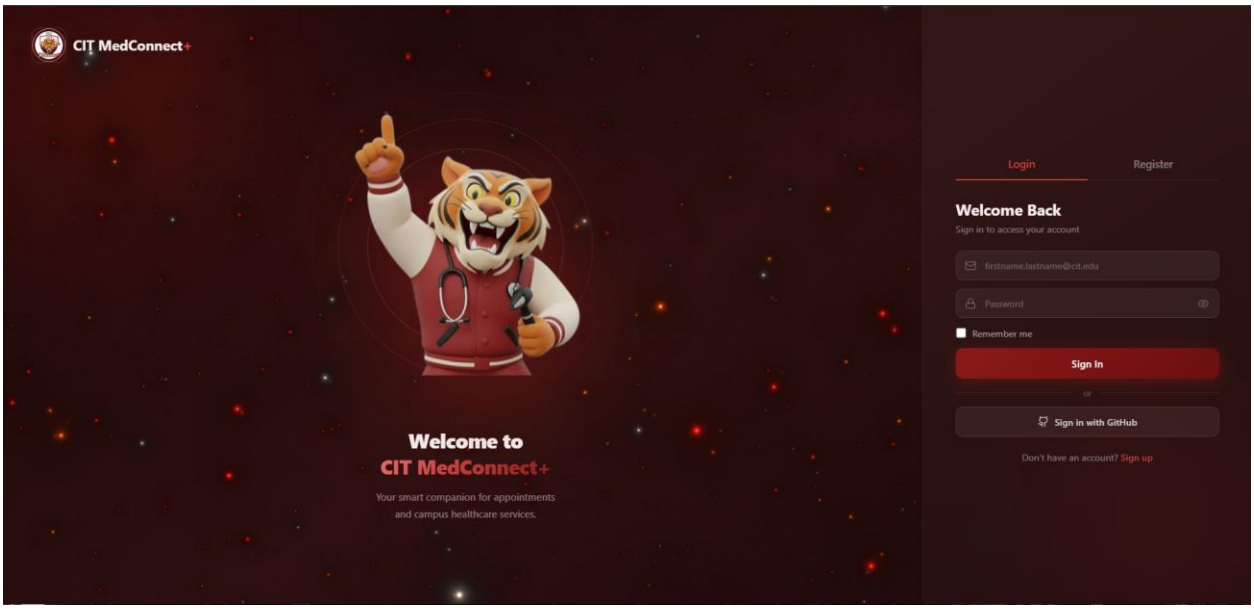
7.1 Design System

Primary Color	Deep Maroon — #8B0000
Accent Color	Gold / Yellow — used for highlights
Background	White #FFFFFF / Light Gray #F5F5F5
Typography	Sans-serif (system default); consistent sizing hierarchy
Navigation (Web)	Left sidebar with icon-based navigation links
Navigation (Mobile)	Bottom navigation bar with Home, Detail, Profile tabs

7.2 Web Application Screens

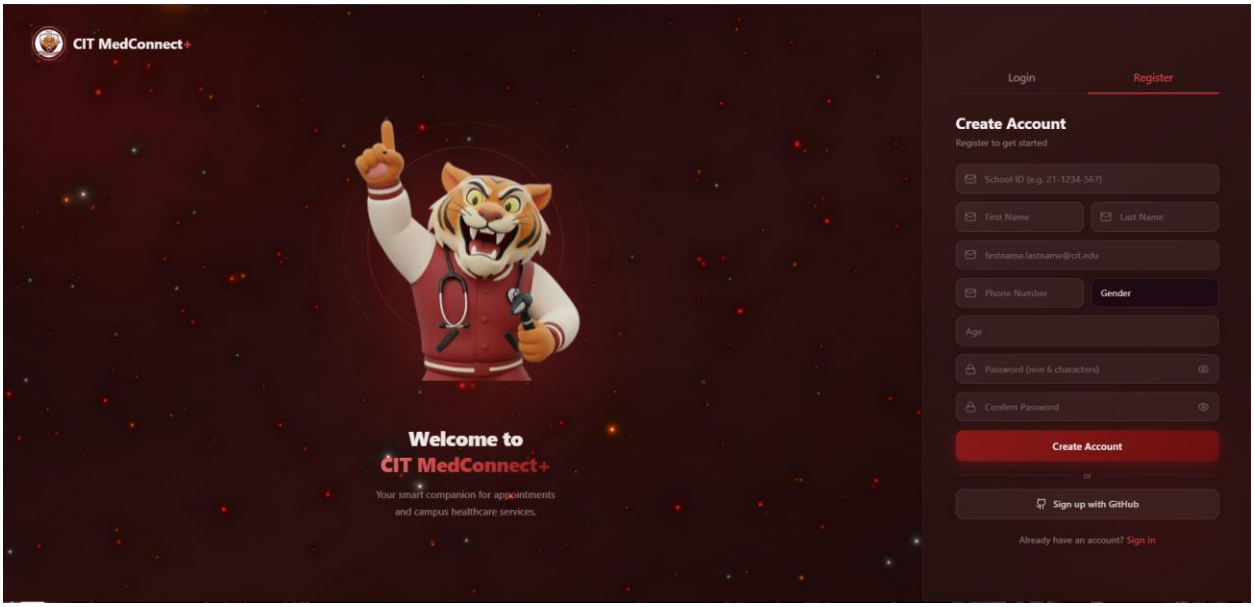
Landing Page / Login

Left panel: marketing copy with CTA buttons. Right panel: sign-in form with email/password fields, remember me toggle, and Sign-Up link.



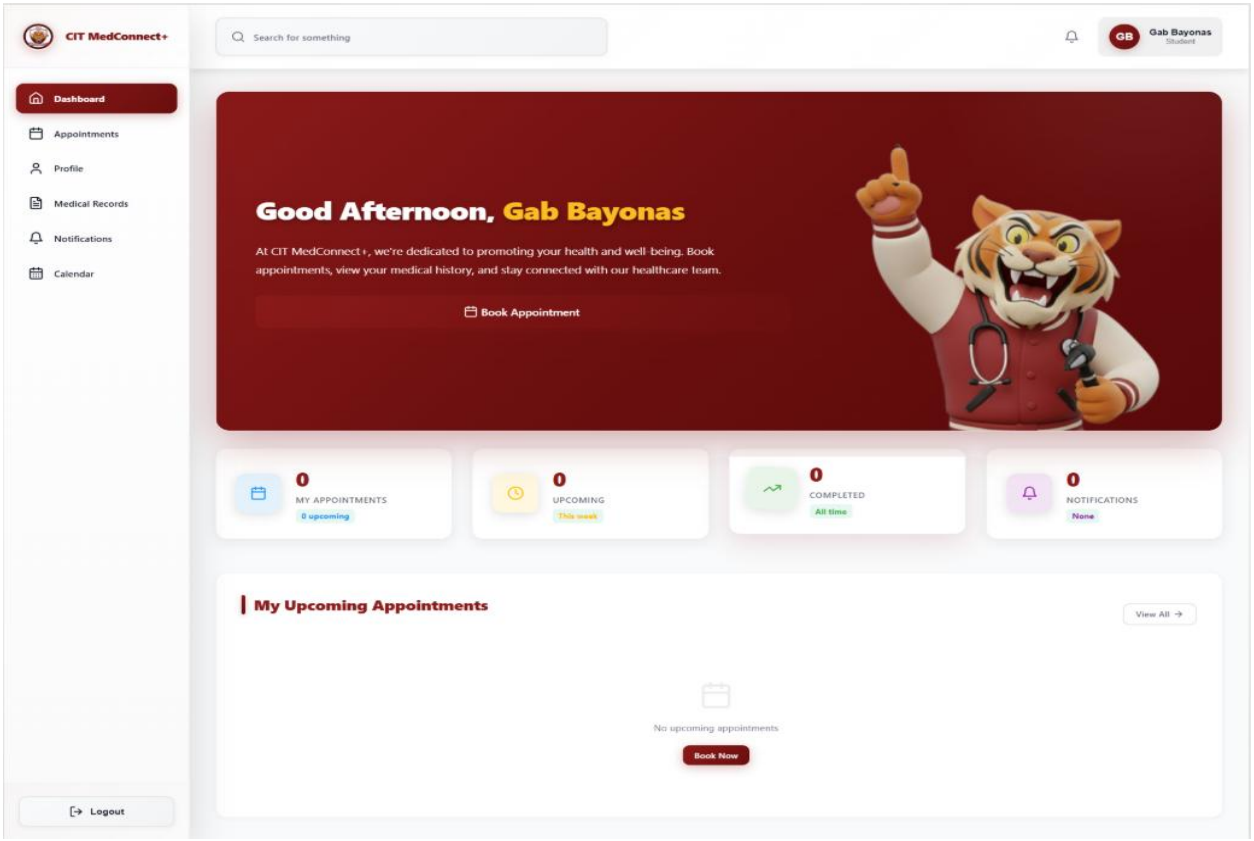
Registration Page

Right panel registration form with institutional email, password, and confirm password fields.



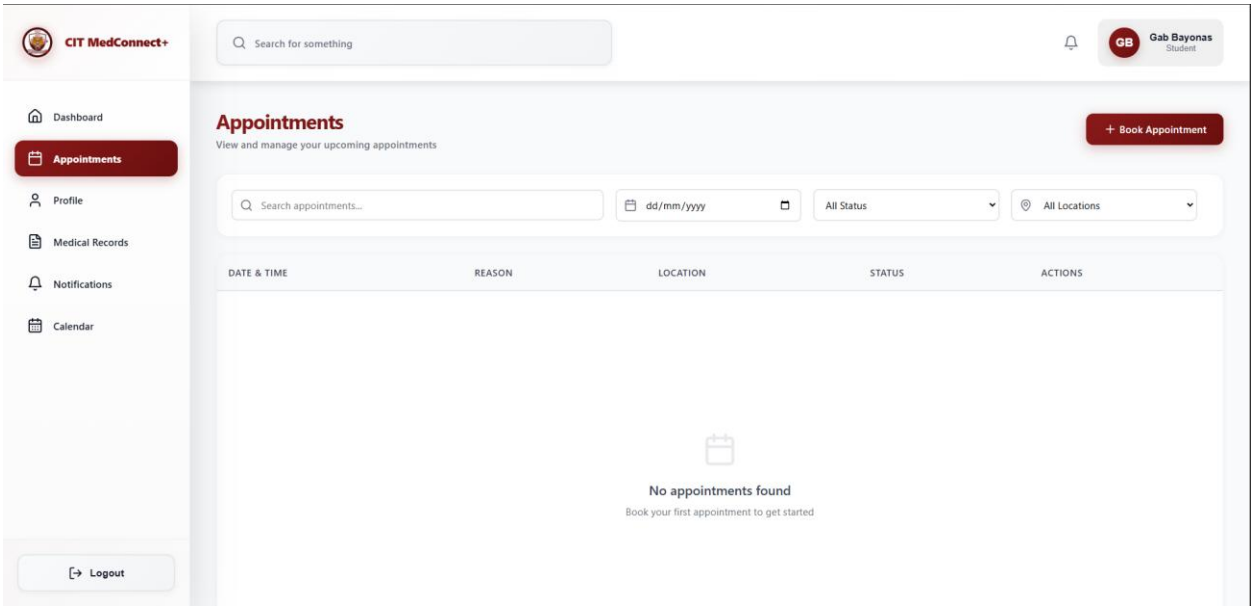
Dashboard

Left sidebar navigation (Home, Appointments, Profile, Reports, Notifications, Messages, Settings). Top bar with search, settings icon, notification bell, and profile picture. Main area shows a welcome panel and quick-action cards.



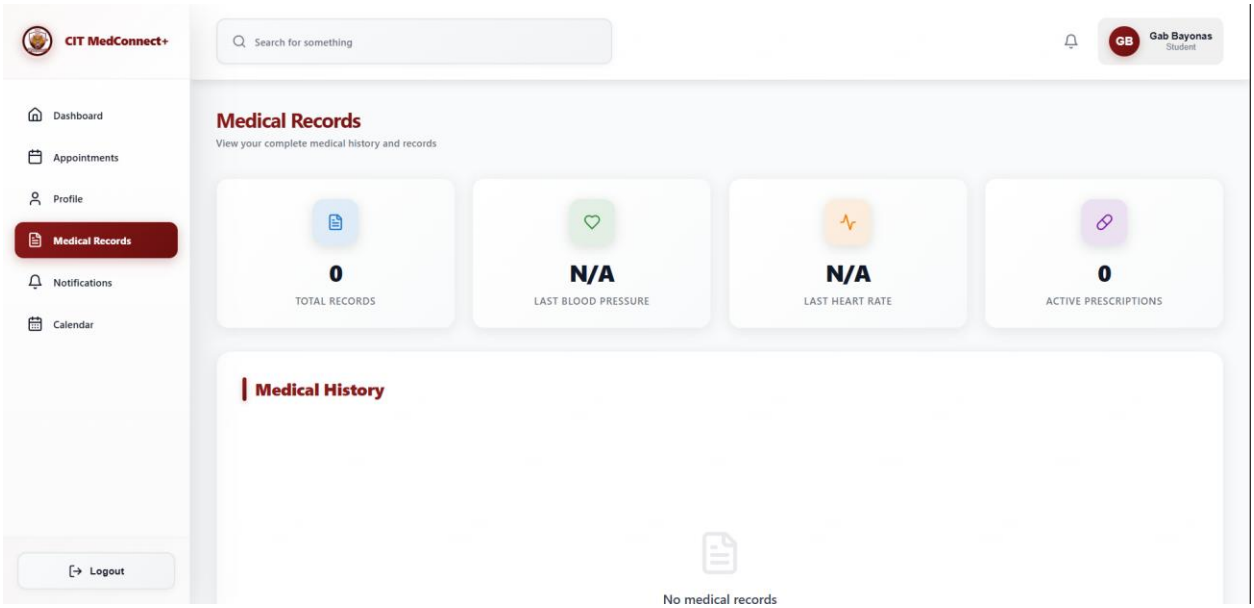
Appointments

Tabular view of time slots for the selected date range. Columns: Time, Patient Diagnosis, Doctor, Patient, Creator, Status, Timer, Actions. Filter controls for date range and status visibility. Staff can confirm, complete, or edit records inline.



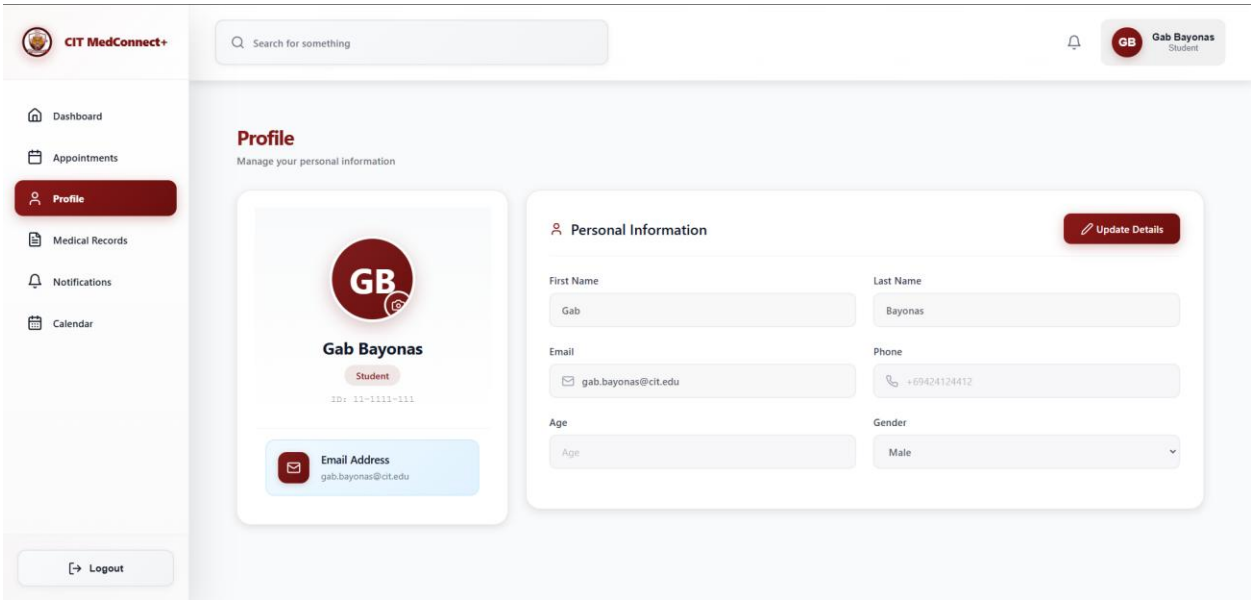
Medical Records

List of medical records with patient name, diagnosis summary, and record date. Staff view includes create and edit actions. Student view is read-only.



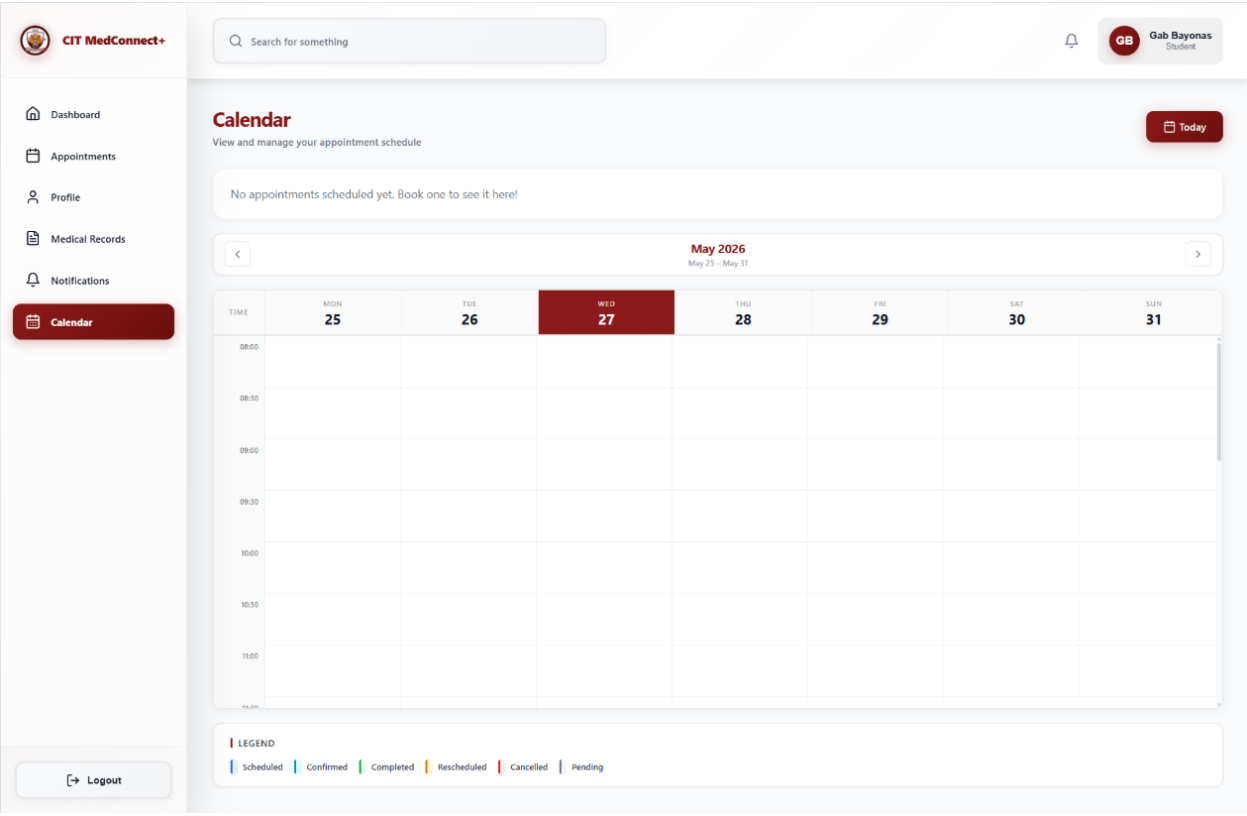
Profile

Displays user profile picture, name, institutional email, full name, ID number, gender, department, age, and timezone. Edit button allows updates. Email address list with option to add additional addresses.



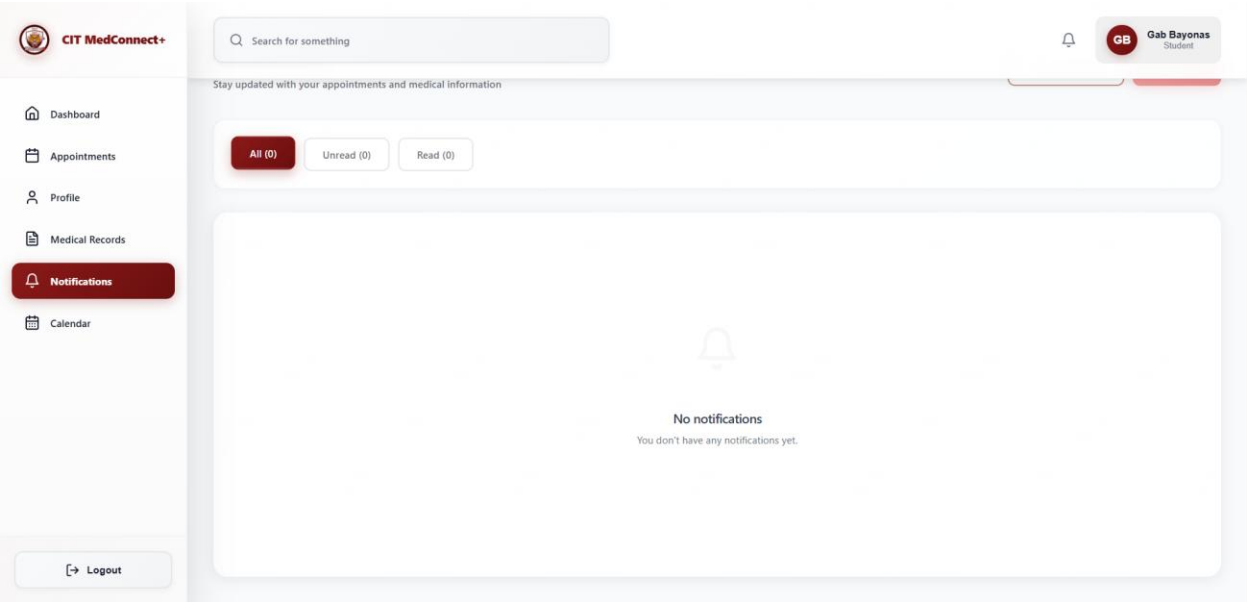
Calendar View

Full-calendar component displaying scheduled appointments. Users can navigate by month and view appointment details on click.



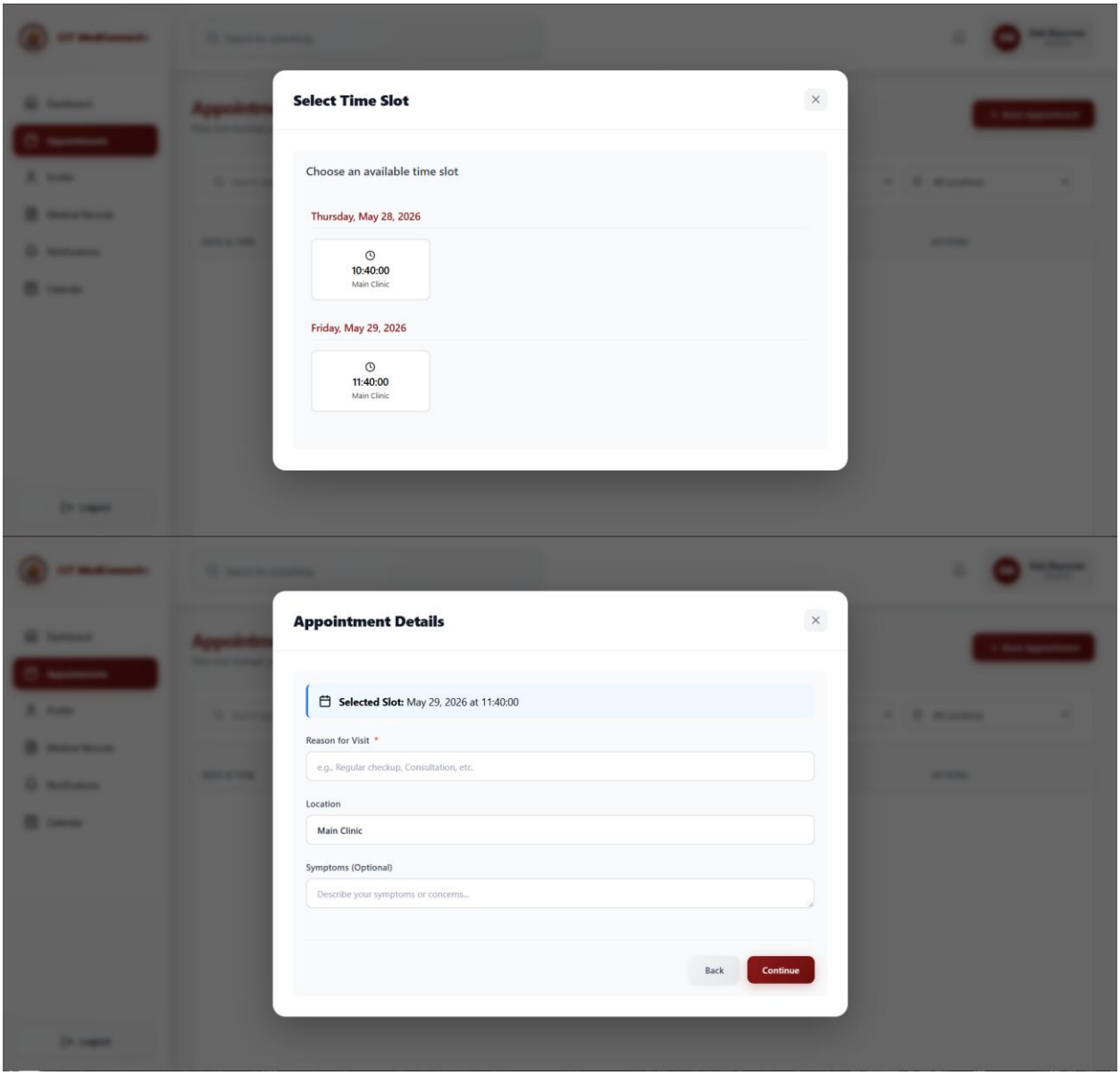
Notifications

Notification list with sender avatar, title, message preview, and timestamp. Mark-as-read on click. View All link for full notification history.



Book Appointment Card

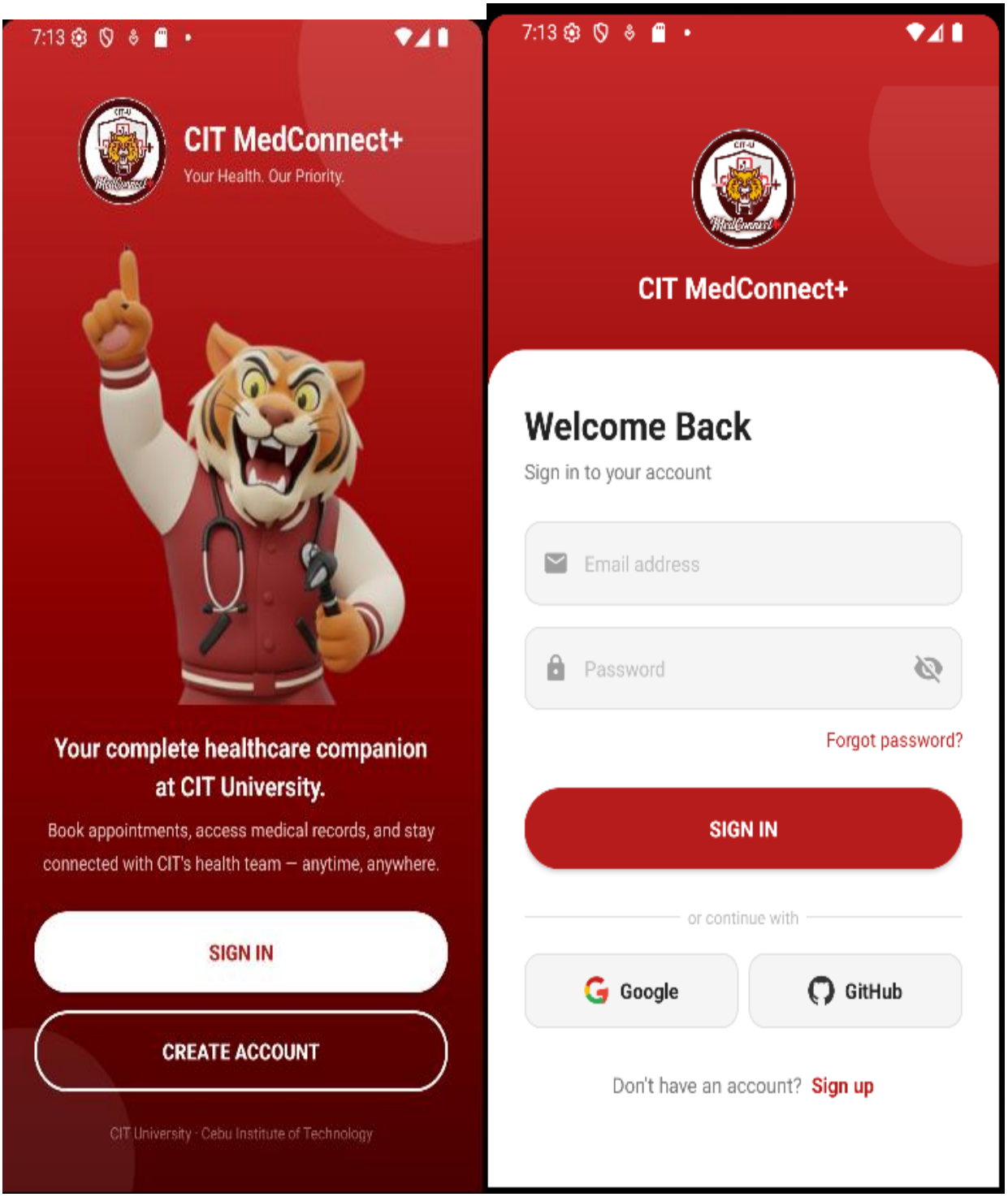
Modal/card form with fields: Full Name (auto-filled), Email Address, Department, Time (available slots dropdown), Date (available days dropdown), and Files (optional upload). Confirm button submits the booking.



7.3 Mobile Application Screens

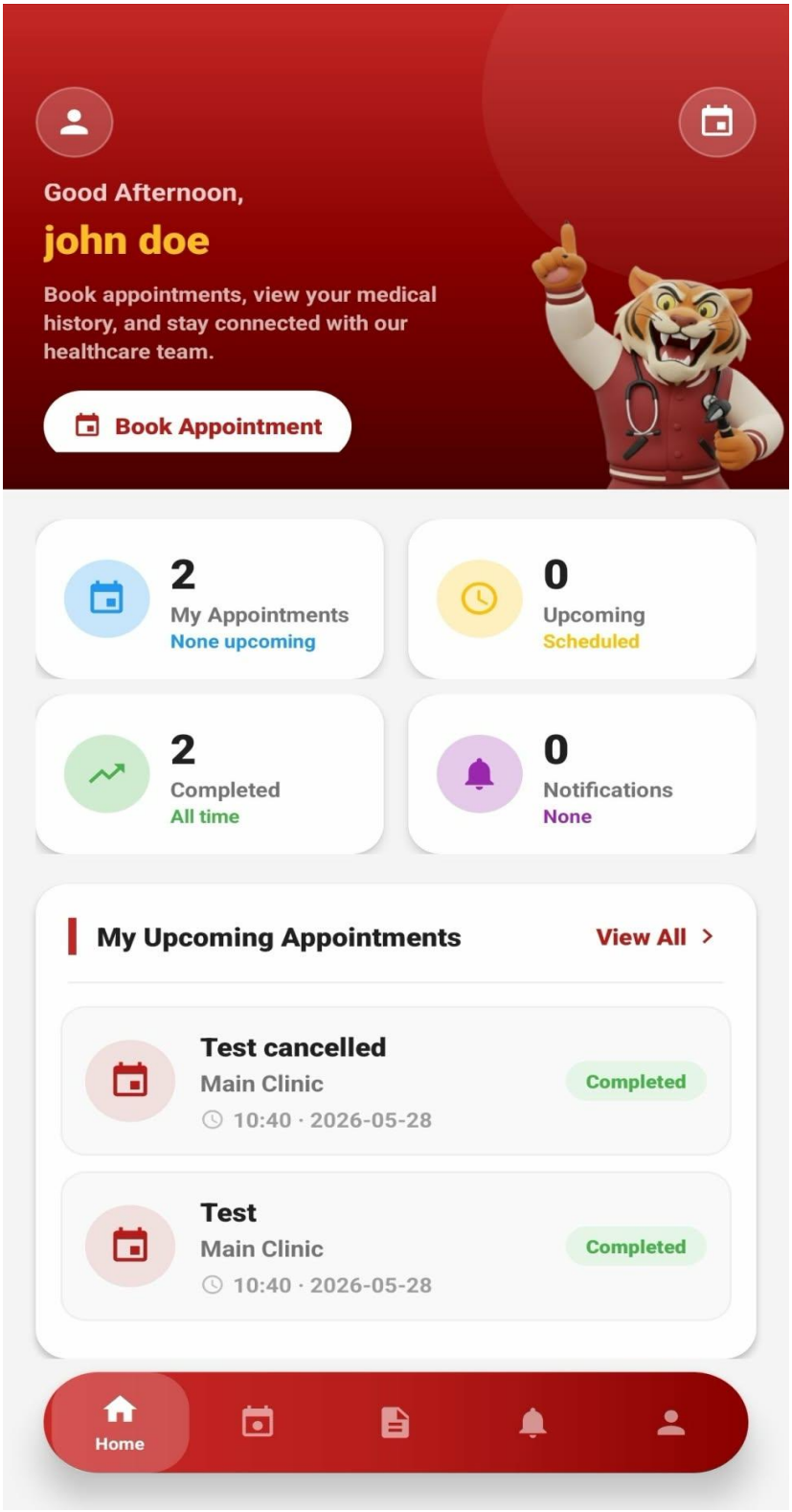
Login Screen

App logo, Sign In heading, Email or Phone Number field, Password field with visibility toggle, Forgot Password link, Sign In button, and Sign Up Free link. Security footer.



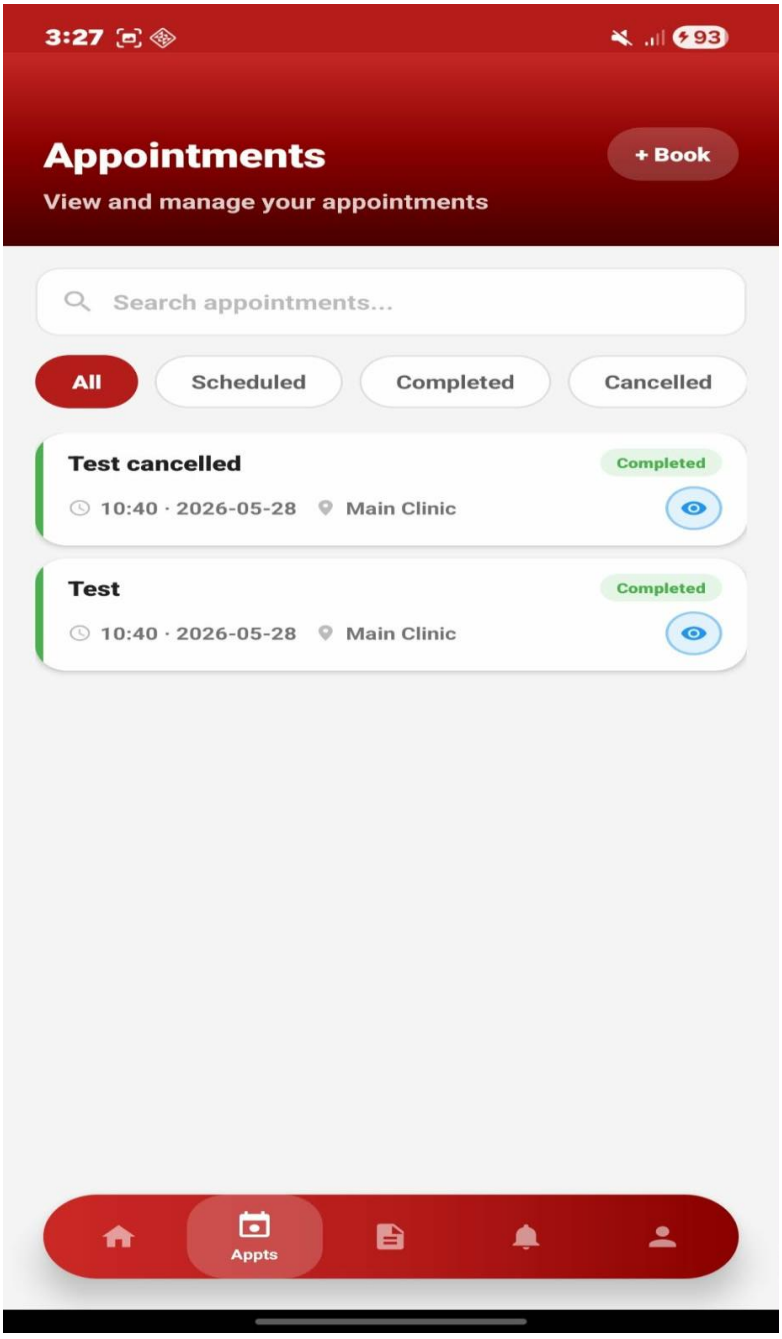
Dashboard Screen

Greeting header, quick-access cards for appointments and medical records, recent notification summary.



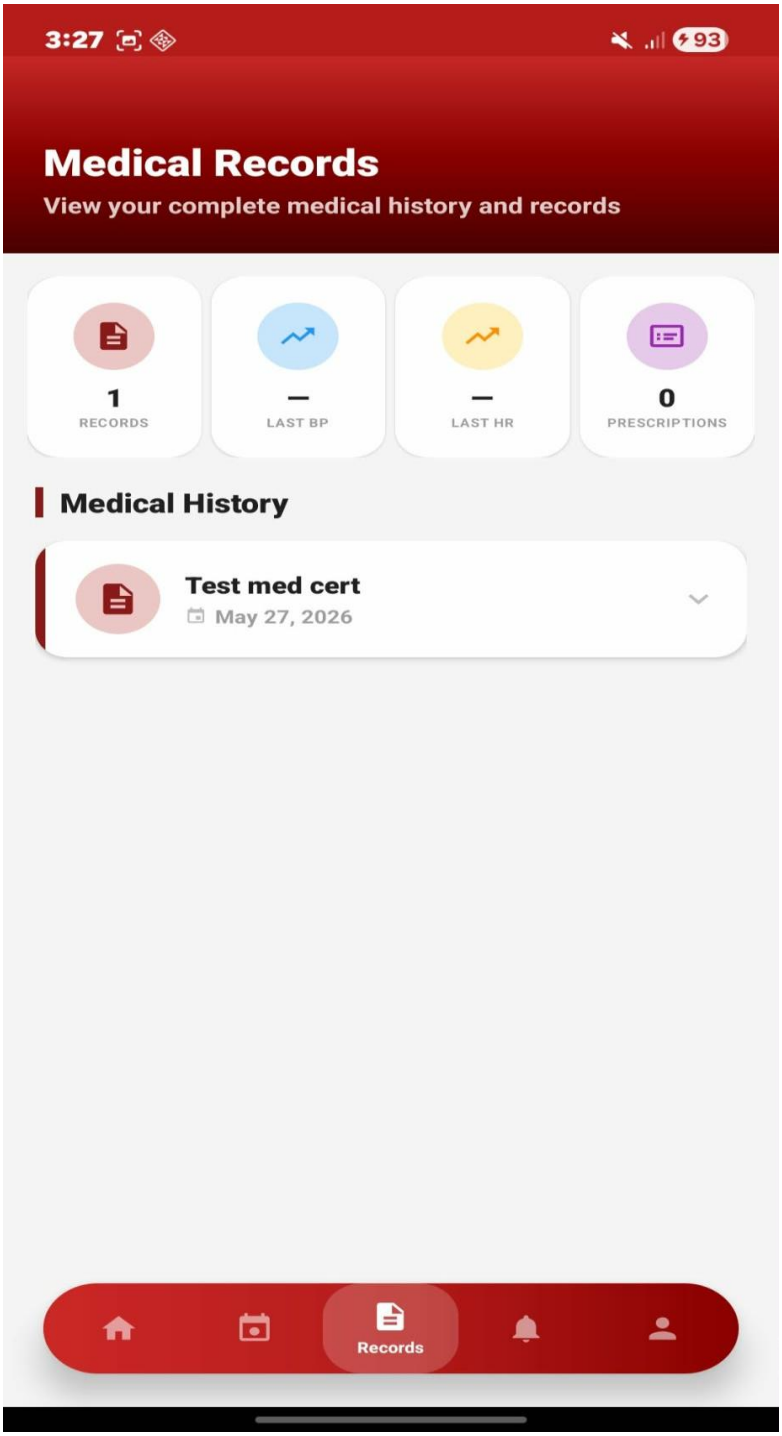
Appointments Screen

List view of upcoming and past appointments with status badges. FAB for creating a new appointment (Student). Staff view shows all appointments with action buttons.



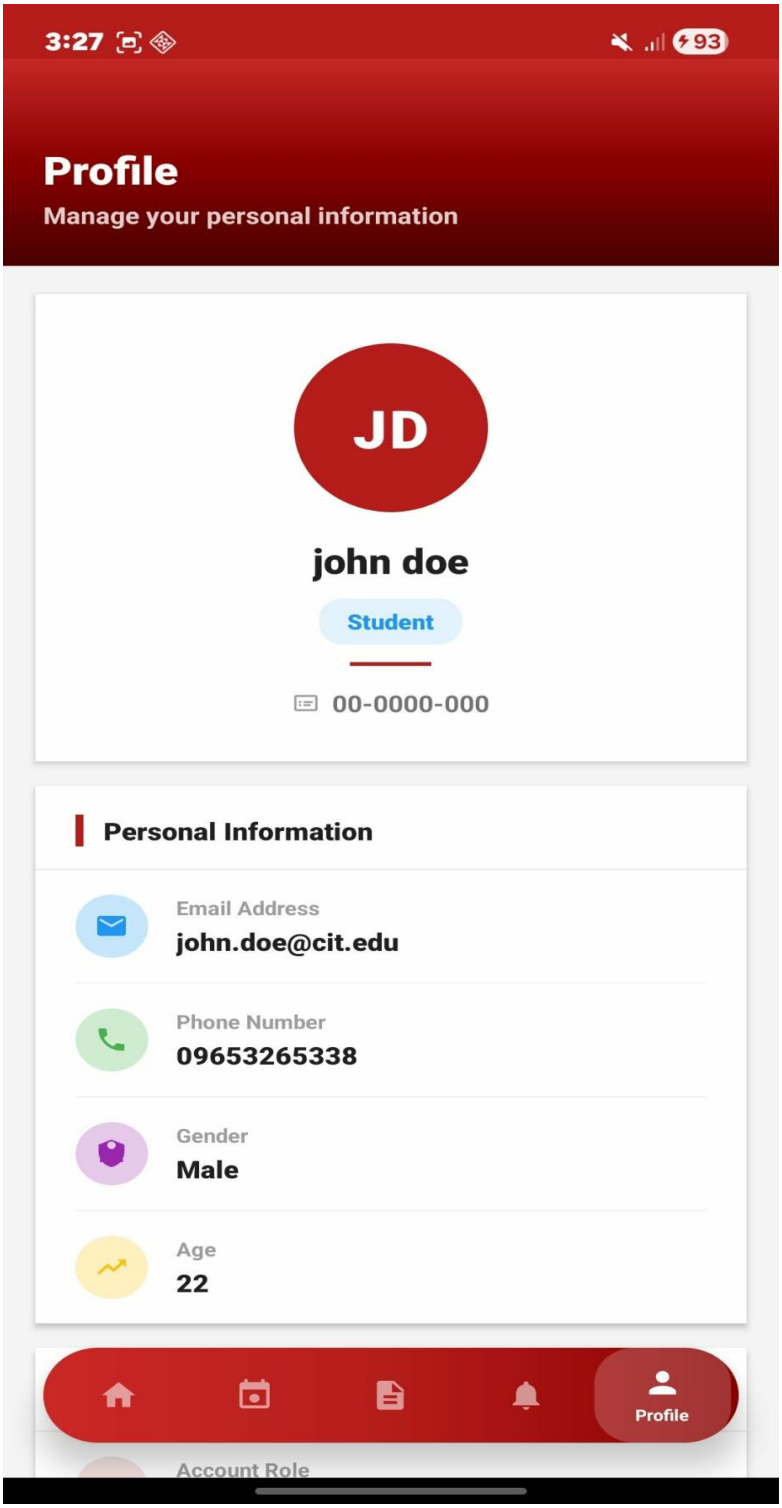
Medical Records Screen

List of medical records for the logged-in student. Tapping a record opens the full detail view. Staff can create and edit records.



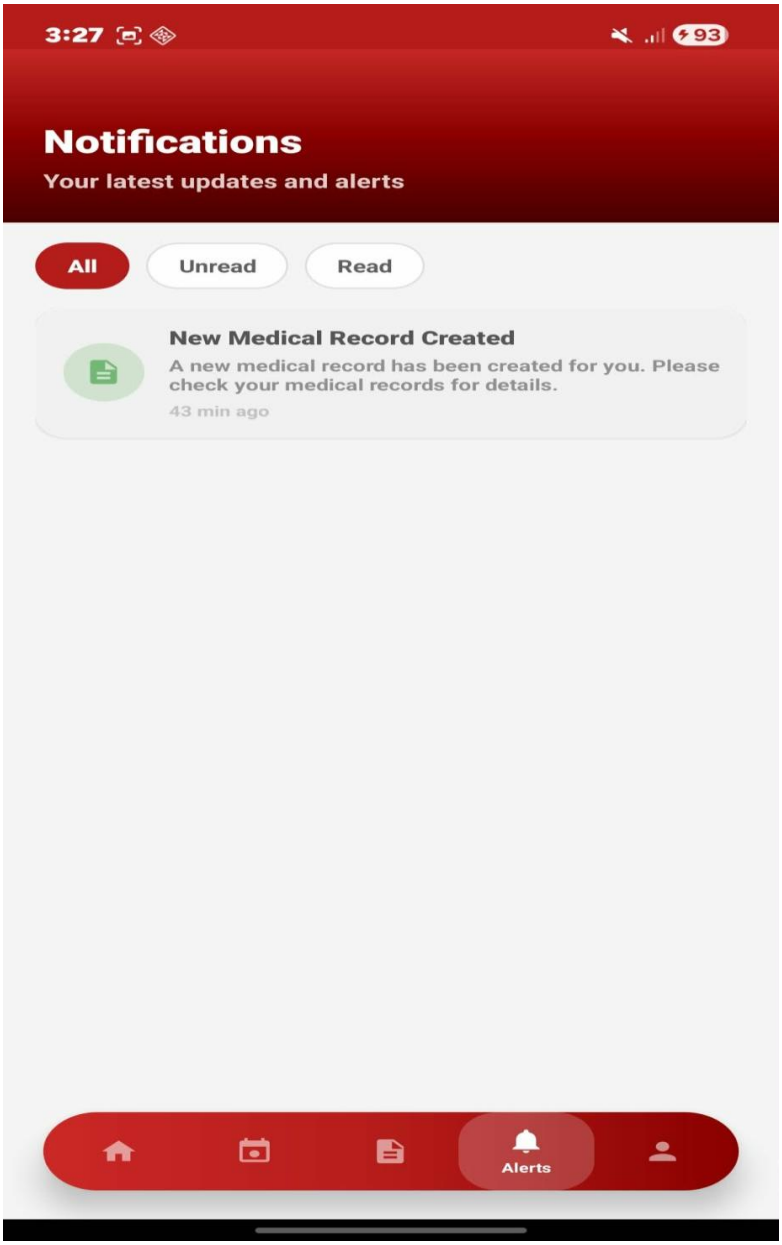
Profile Screen

Profile header with avatar and role badge. Statistics row (appointments, records). Settings sections: Notifications, Privacy & Security, and Support. Sign Out button.



Notifications Screen

List of in-app notifications with type icon, title, message, and relative timestamp. Mark-all-read action.



8.0 PROJECT PLAN

8.1 Project Timeline

Phase	Key Deliverables	Duration	Status
Phase 1: Planning & Design	SDD, FRS, ERD, API spec, wireframes	Week 1–2	Completed
Phase 2: Backend Development	Spring Boot API, JPA entities, security config, all endpoints	Week 3–4	Completed
Phase 3: Web Frontend	React app with all pages, API integration, GitHub OAuth callback	Week 5–6	Completed
Phase 4: Mobile Application	Android Activities, ViewModels, Retrofit integration, DataStore	Week 7–8	Completed
Phase 5: Integration & Deployment	End-to-end testing, deployment, final documentation	Week 9–10	In Progress

8.2 Milestones

Milestone	Description	Target
M1	All design documents completed (SDD, FRS, ERD, Wireframes)	End of Week 2
M2	Backend API fully functional and tested via Postman	End of Week 4
M3	Web application feature-complete with API integration	End of Week 6
M4	Android application feature-complete and tested on device	End of Week 8
M5	Full system deployed and integration-tested	End of Week 10

8.3 Risk Mitigation

- Start with the minimal working version of each module and iterate.
- Test authentication and RBAC early to unblock all dependent features.
- Perform frequent Supabase database backups during active development.
- Validate all API endpoints using Postman before frontend integration.
- Prioritize core healthcare workflows (authentication, appointment booking, medical records) before UI polish.
- Use feature branches in Git to isolate changes and reduce merge conflicts.

8.4 Critical Path

37. User authentication and token management (Week 3) — required by all downstream features.
38. Time slot and appointment booking API (Week 4) — core patient workflow.
39. Role-based access control enforcement (Week 4) — security prerequisite for all protected endpoints.
40. Medical record creation linked to appointments (Week 4) — key staff workflow.
41. Frontend integration for web and mobile (Weeks 5–8) — dependent on completed API.
42. End-to-end integration testing (Week 9) — validates full system cohesion before deployment.